

Java LinkedList

A Comprehensive Technical Guide to Mastering
Dynamic Linear Data Structures

CodeHive Academy

1. Introduction to LinkedList

In Java, the `LinkedList` class is part of the **Java Collections Framework** found in the `java.util` package. While it provides functionality similar to an `ArrayList`, its internal architecture is fundamentally different, making it ideal for specific performance-critical operations.

Basic Implementation

The following example demonstrates how to define and populate a simple `LinkedList` of Strings:

```
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        // Initialization
        LinkedList<String> cars = new LinkedList<String>();

        // Adding elements
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");

        // Displaying the list
        System.out.println(cars);
    }
}
```

Unlike standard arrays, the `LinkedList` does not require an initial size. It grows and shrinks dynamically as elements are added or removed.

2. ArrayList vs. LinkedList

Both classes implement the `List` interface, meaning you can use the same methods (`add`, `get`, `remove`, `size`) on both. However, their physical memory layout determines their efficiency.

The ArrayList (Contiguous Memory)

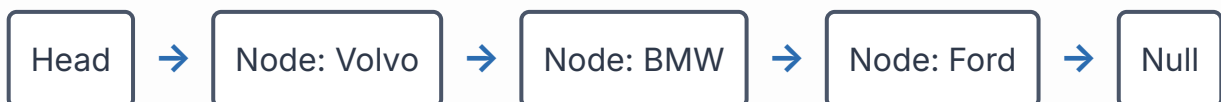
An `ArrayList` uses a dynamic array internally. Elements are stored in contiguous memory locations. When the array is full and a new element is added, Java creates a larger array (usually 50% bigger) and copies all existing elements over.

The LinkedList (Non-Contiguous Memory)

A `LinkedList` stores elements in independent "nodes." Each node contains two parts:

1. **Data:** The actual value stored (e.g., a String or Object).
2. **Next Reference:** A pointer (memory address) to the next node in the sequence.

LinkedList Logic Visualized:



3. Performance Analysis (Big O)

Understanding the complexity of operations is key to choosing the right structure. In computer science, we use Big O notation to describe this efficiency.

Operation	ArrayList	LinkedList	Why?
Random Access (get by index)	$O(1)$	$O(n)$	LinkedList must traverse from the start.
Insert/Delete (Start)	$O(n)$	$O(1)$	ArrayList must shift all other elements.
Insert/Delete (End)	$O(1)$ amortized	$O(1)$	Both handle ends efficiently.
Search	$O(n)$	$O(n)$	Both require linear search.

Key Rule: Use an **ArrayList** for storing and accessing data where "gets" are frequent. Use a **LinkedList** when you need to frequently manipulate (insert/delete) data, especially at the beginning of the list.

4. Specialized LinkedList Methods

While standard List methods work, the LinkedList class provides unique methods because it also implements the **Deque** (Double Ended Queue) interface.

Method	Description
<code>addFirst()</code>	Adds an element at the very beginning.
<code>addLast()</code>	Adds an element at the end.
<code>removeFirst()</code>	Removes and returns the first element.
<code>removeLast()</code>	Removes and returns the last element.
<code>getFirst()</code>	Retrieves the first element without removing it.
<code>getLast()</code>	Retrieves the last element without removing it.

```
LinkedList<String> queue = new LinkedList<>();
queue.addFirst("Priority Task"); // Push to front
queue.addLast("Normal Task");   // Push to back

String first = queue.getFirst(); // "Priority Task"
queue.removeLast();              // Removes "Normal Task"
```

5. Modern Syntax: The `var` Keyword

Introduced in **Java 10**, Local Variable Type Inference allows you to skip explicit type declarations for local variables. The compiler infers the type from the constructor.

Traditional vs. Modern Declaration

```
// Prior to Java 10
LinkedList<String> cars = new LinkedList<String>();

// Java 10+ (Using 'var')
var cars = new LinkedList<String>();
```

Benefits of `var`:

- Reduces verbosity in code.
- Maintains strong typing (the variable is still a `LinkedList`).
- Useful when the generic type names are very long.

6. Coding to Interfaces

You will often see Java developers declare a list using the `List` interface rather than the concrete class. This is an application of **Polymorphism**.

```
import java.util.List;
import java.util.LinkedList;

List<String> list = new LinkedList<>();
```

Why use the List Interface?

1. **Flexibility:** If you decide later that an `ArrayList` would be better for performance, you only need to change one line: `new ArrayList<>();`.
2. **Abstraction:** Your methods can accept a `List<T>` parameter, allowing them to work with any list implementation (`ArrayList`, `LinkedList`, `Vector`, etc.).

7. Memory Overhead Comparison

One downside of `LinkedList` that is rarely mentioned in basic tutorials is **Heap Memory Overhead**.

In an `ArrayList`, the memory is mostly consumed by the data itself. However, in a `LinkedList`, every single item requires a `Node` object wrap. A `Node` usually consumes extra memory to store:

- A reference to the data.
- A reference to the next Node.
- A reference to the previous Node (since Java's `LinkedList` is Doubly Linked).

On a 64-bit JVM, these references can add up significantly. For a large list of integers, a `LinkedList` can use significantly more memory than an `ArrayList`.

8. Real World Scenario: The Undo Buffer

A classic use case for a `LinkedList` is the "Undo" functionality in a text editor. Every time you type, a new "state" is added to the end of the list. If the user hits undo, we simply `removeLast()`.

```
var undoHistory = new LinkedList<String>();

// User types something
undoHistory.addLast("State 1: Hello");
undoHistory.addLast("State 2: Hello World");

// User hits Undo
if (!undoHistory.isEmpty()) {
    String lastState = undoHistory.removeLast();
    System.out.println("Reverting change: " + lastState);
}
```

9. Final Summary

Choose your data structure wisely based on your application needs:

- **Use `LinkedList` when:**

- You have many insertions or deletions at the start or end.
- You are implementing a Queue or a Stack.
- The list size is small or memory is not a major constraint.

- **Use `ArrayList` when:**

- You need frequent random access to elements (e.g., `list.get(500)`).
- Memory efficiency is a priority.
- You are mostly adding elements to the end of the list.

© 2026 CodeHive Academy. All rights reserved.
Java Collections Documentation Series - Issue #04